INSTITUT TEKNOLOGI BANDUNG
**PROGRAM STUDI TEKNIK TELEKOMUNIKASI**
JALAN GANESHA NO. 10 Gedung Labtek 8 Lantai 4 ☎ (022)2500962, (022)2501661
BANDUNG 40132

## Dokumentasi Produk Tugas Akhir

# Lembar Sampul Dokumen

| | |
|---|---|
| Judul Dokumen | **TUGAS AKHIR TEKNIK TELEKOMUNIKASI:** *Judul tugas akhir: Design and Implementation of In-Kernel Fast Path Data Plane Using eBPF XDP for High-Performance Networking* |
| Jenis Dokumen | **PROPOSAL PENGEMBANGAN DESAIN AWAL** |

Catatan: Dokumen ini dikendalikan penyebarannya oleh Prodi Teknik Telekomunikasi ITB

| | |
|---|---|
| Nomor Dokumen | **T30-01-TA2025.12.12** |
| Nomor Revisi | **1** |
| Nama File | **T30** |
| Tanggal Penerbitan | **12 Desember 2025** |
| Unit Penerbit | **Prodi Teknik Telekomunikasi – ITB** |
| Jumlah Halaman | 17 |

| Data Pengusul | | | | |
|---|---|---|---|---|
| Pengusul | Nama | Athala Rania Insyra | Jabatan | Anggota Kelompok |
| | NIM | 18122038 | Tanda Tangan | |
| | Nama | Muhammad Furqan Riansyah Putra | Jabatan | Anggota Kelompok |
| | NIM | 18122026 | Tanda Tangan | |
| Pembimbing 1 | Nama | Dr.-Ing. Chairunnisa, S.T, M.T. | Tanda Tangan | |
| | NIP | 197702162008012017 | | |
| Pembimbing 2 | Nama | Dr. Ing. Eueung Mulyana, S.T, M.Sc. | Tanda Tangan | |
| | NIP | 197412182008011009 | | |
| Lembaga<br>Program Studi Teknik Telekomunikasi Sekolah Teknik Telekomunikasi dan Informatika Institut Teknologi Bandung | | | | |
| Alamat<br>Labtek 8, Lantai 4, Jalan Ganesha no. 10, Bandung | | | | |
| Telepon : +62 22 250 0962, +62 22 250 1661 | Faks :+62 22 250 0962 | | Email:stei@stei.itb.ac.id | |



© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh Isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.

DAFTAR ISI

DAFTAR ISI

1. PENGANTAR
   1.1 Ringkasan Isi Dokumen
   1.2 Tujuan Penulisan dan Aplikasi/Kegunaan Dokumen
   1.3 Daftar Singkatan

2. ALTERNATIF SOLUSI DESAIN
   2.1 Alternatif Metode Desain
      2.1.1. Subsistem ke-1
      2.1.2. Subsistem ke-2
      2.1.3. Subsistem ke-N
      2.1.4. Sistem Terintegrasi
   2.2 Estimasi Biaya Produk
   2.3 Analisis Finansial
   2.4. Penentuan Solusi Desain Terbaik
      2.4.1. Pembobotan Kategori dari Alternatif Metode Desain
      2.4.2. Matriks Keputusan

3. RENCANA PELAKSANAAN PROYEK DESAIN
   3.1. Sumber dan Cara Mendapatkan Dana Pembuatan Desain
   3.2. Struktur Organisasi Pengembangan dan Pemasaran Desain

4. KESIMPULAN DAN RENCANA SELANJUTNYA
   4.1 Kesimpulan
   4.2 Rencana Desain Selanjutnya (Tugas Akhir 2)

Referensi



© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.

# 1 PENGANTAR

## 1.1 Ringkasan Isi Dokumen

Penelitian ini berfokus pada perancangan dan implementasi *In-Kernel Fast Path Data Plane* menggunakan teknologi eBPF/XDP untuk mengatasi permasalahan *bottleneck* performa pada infrastruktur jaringan berkecepatan tinggi, sekaligus membangun *dashboard* otomatisasi yang berfungsi sebagai platform terpusat untuk orkestrasi, *monitoring*, dan visualisasi hasil eksperimen. Sistem ini dirancang sebagai solusi alternatif yang menyeimbangkan kebutuhan *throughput* tinggi dan efisiensi sumber daya, mengatasi inefisiensi mekanisme *busy-polling* pada *framework kernel bypass* berbasis *user-space* seperti DPDK dan VPP, tanpa menghilangkan integritas fitur keamanan kernel.

Penelitian ini secara khusus akan mengembangkan logika pemrosesan paket *native* menggunakan *library* LibBPF dan mengintegrasikannya dengan sistem orkestrasi otomatis berbasis Python. Evaluasi desain difokuskan pada pencapaian target teknis yang terukur, yaitu pencapaian *throughput* minimal 55% dari performa DPDK, penggunaan CPU yang proporsional saat *idle* (< 5%), serta pemangkasan waktu konfigurasi *testbed* hingga 78% melalui mekanisme otomasi. Pengujian ini bertujuan untuk memvalidasi kelayakan teknis eBPF/XDP sebagai solusi yang stabil dan efisien untuk lingkungan jaringan modern.

Persyaratan desain diformulasikan melalui analisis komparatif terhadap berbagai alternatif metode teknis, mempertimbangkan aspek performa, fleksibilitas, dan kompleksitas implementasi. Berdasarkan pembobotan matriks keputusan, ditetapkan solusi desain terbaik (Alternatif 18) yang mengombinasikan *framework* React.js dan Grafana untuk visualisasi data, LibBPF berbasis C/Native untuk pengembangan program eBPF yang portabel, serta Python Fabric untuk orkestrasi otomatisasi *testbed*. Kombinasi ini dipilih untuk memenuhi objektif efisiensi performa dan kemudahan manajemen operasional.

Struktur sistem terdiri dari tiga subsistem utama: (1) Dashboard Otomatisasi dan Visualisasi, (2) Subsistem Eksperimen eBPF/XDP, dan (3) Subsistem Otomasi Testbed Multi-Node. Ketiga subsistem ini bekerja secara terpadu untuk mendukung pelaksanaan eksperimen jaringan secara otomatis dan konsisten. Selain aspek teknis, analisis finansial menunjukkan bahwa proyek ini memiliki kelayakan ekonomi yang tinggi dengan estimasi *Break-Even Point* sekitar 15 bulan, menjadikannya solusi yang siap untuk direalisasikan dan diuji kinerjanya pada tahap Tugas Akhir 2.

## 1.2 Tujuan Penulisan dan Aplikasi/Kegunaan Dokumen

Tujuan penulisan dokumen ini adalah sebagai berikut:

- Mendokumentasikan proses penentuan solusi desain terbaik melalui analisis perbandingan dan matriks keputusan terhadap berbagai alternatif metode teknis pada subsistem dashboard, eksperimen eBPF, dan otomasi.
- Menyajikan analisis kelayakan ekonomi melalui estimasi biaya investasi riil dan perhitungan BEP guna membuktikan bahwa desain sistem ini *feasible* untuk direalisasikan dan memiliki nilai keberlanjutan.
- Memberikan panduan rencana kerja untuk tahap selanjutnya (Tugas Akhir 2), yang mencakup strategi implementasi teknis, integrasi sistem terpusat, serta skenario pengujian performa (*benchmarking*) yang akan dilakukan.


© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.

Dokumen ini ditujukan kepada dosen pembimbing tugas akhir dan tim tugas akhir Program Studi Teknik Telekomunikasi ITB sebagai bahan penilaian tugas akhir dan acuan pelaksanaan pengembangan sistem.

### 1.3 DAFTAR SINGKATAN

| SINGKATAN | ARTI |
|---|---|
| **NIC** | *Network Interface Card* |
| **CPU** | *Central Processing Unit* |
| **PC** | *Personal Computer* |
| **VPS** | *Virtual Private Server* |
| **VPP** | *Vector Packet Processing* |
| **DPDK** | *Data Plane Development Kit* |
| **eBPF** | *Extended Berkeley Packet Filter* |
| **XDP** | *eXpress Data Path* |
| **DTRR** | *Deployment Time Reduction Rate* |
| **BEP** | *Break-Even Point* |
| **SaaS** | *Software-as-a-Service* |

## 2. ALTERNATIF SOLUSI DESAIN

### 2.1. Alternatif Metode Desain

Bagian ini menguraikan usulan alternatif metode teknis untuk merealisasikan platform eksperimen *in-kernel fast path* berbasis eBPF/XDP serta dashboard otomatisasi sesuai dengan persyaratan desain yang telah ditetapkan. Pemilihan metode ini difokuskan untuk memenuhi objektif efisiensi performa (*throughput* dan utilisasi CPU) serta otomatisasi penuh pada manajemen *testbed*.

Pemetaan dari persyaratan desain menjadi alternatif metode implementasi diperlihatkan secara rinci pada Tabel 1. Proses penentuan metode dilakukan dengan menganalisis relevansi teknis pada tiga fungsi utama sistem, yaitu:

1. Subsistem Dashboard Otomatisasi dan Visualisasi
2. Subsistem Eksperimen eBPF/XDP
3. Subsistem Orkestrasi dan Otomasi Testbed Multi-Node

Berdasarkan analisis terhadap spesifikasi fungsional (seperti target *DTRR* ≥ 78% dan *throughput* ≥ 55%) serta *constraint* desain pada kolom 2 dan 3, selanjutnya ditetapkan kategori metode yang relevan, meliputi *framework* visualisasi, *toolchain* pengembangan eBPF, dan alat otomasi konfigurasi. Akhirnya, untuk setiap kategori tersebut, diajukan beberapa kandidat alternatif teknologi untuk dievaluasi pada tahap selanjutnya.

Tabel 1. Pemetaan Persyaratan Desain menjadi Alternatif Metode Desain

| **Fungsi** | **Relevansi Fungsi** | | **Metode** | |
|---|---|---|---|---|
| | **Persyaratan dan Spesifikasi Fungsional** | **Subobjektif dan Constraint, beserta Syarat Pemenuhanannya** | **Kategori** | **Alternatif** |


© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.

| | | | | |
|---|---|---|---|---|
| Subsistem Dashboard Otomatisasi dan Visualisasi | **Rangkuman persyaratan fungsional 1,4:**<br>Menyediakan fitur inventarisasi hasil (DPDK, VPP, XDP/eBPF), manajemen/kontrol eksperimen, dan visualisasi hasil terpusat.<br>**Rangkuman spesifikasi:**<br>Dashboard menampilkan minimal 3 metrik utama (throughput, latensi, CPU usage) dan status node secara *real-time* (update < 1 detik). | **Rangkuman subobjektif 4 dan constraint**:<br>Menyediakan visualisasi hasil eksperimen dan metrik performa yang informatif.<br>**Rangkuman syarat pemenuhan:**<br>Dashboard menampilkan 3 metrik utama secara akurat dan sinkron dengan data aktual log eksperimen. | Framework Frontend dan Visualisasi | 1. React.js<br>2. Grafana<br>3. React.js dan Grafana |
| Subsistem Eksperimen eBPF/XDP | **Rangkuman persyaratan fungsional 2:**<br>Mampu menjalankan eksperimen eBPF/XDP di testbed dan mencatat hasil metrik performa secara mandiri.<br>**Rangkuman spesifikasi:**<br>Menjalankan program eBPF/XDP, mencapai *throughput* ≥55% dari baseline DPDK/VPP, mendukung filtering firewall, dan observasi metrik kernel. | **Rangkuman subobjektif 1,2 dan constraint:**<br>Implementasi *in-kernel fast path* dengan overhead pemrosesan minimal (tanpa *busy-polling* 100%).<br>**Rangkuman syarat pemenuhan:**<br>Throughput ≥55% DPDK/VPP, CPU usage < 5% saat *idle* (0 Mpps), dan *filtering packet* firewall berhasil dilakukan. | Library/Toolchain Pengembangan eBPF | 1. BCC (BPF Compiler Collection)<br>2. LibBPF (C/Native Library) |
| Subsistem Orkestrasi dan Otomasi Testbed Multi-Node | **Rangkuman persyaratan fungsional 3:**<br>Mengatur *deployment* otomatis (instalasi OS, konfigurasi jaringan, dependensi) dan menjalankan *pipeline* eksperimen tanpa intervensi manual.<br>**Rangkuman spesifikasi:**<br>*Pipeline* otomasi mampu menjalankan seluruh tahapan *end-to-end*, dan | **Rangkuman subobjektif 3 dan constraint:**<br>Otomatisasi penuh terhadap *deployment* dan manajemen eksperimen untuk menjamin *repeatability*.<br>**Rangkuman syarat pemenuhan:**<br>Seluruh tahapan dapat dijalankan dengan satu perintah, | Tools Otomasi Konfigurasi & Orkestrasi | 1. Ansible<br>2. Shell Scripting (Bash)<br>3. Python Scripts (Fabric) |


© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.

| | | | | |
|---|---|---|---|---|
| | 78% lebih cepat dari konfigurasi manual [3]. | dan mencapai *Deployment Time Reduction Rate* (DTRR) ≥ 78% [3]. | | |

### 2.1.1. Subsistem Dashboard Otomatisasi dan Visualisasi

Pada subsistem ini, metode yang dipilih berperan vital dalam memenuhi Subobjektif 4 dan Persyaratan Fungsional 1 & 4. Berikut adalah analisis alternatif metode yang diajukan:

**Alternatif framework 1: React.js**

React.js adalah *library* JavaScript bersifat *open-source* yang dikembangkan oleh Meta (Facebook) untuk membangun UI berbasis komponen [1]. React.js menggunakan pendekatan deklaratif dan *Virtual DOM* untuk merender tampilan aplikasi web yang dinamis secara efisien [2].

Keunggulan framework React.js adalah:

1. **Fleksibilitas Tinggi:** React.js memungkinkan kustomisasi yang sangat luas karena berbasis JavaScript murni, memungkinkan pembuatan logika komponen interaktif yang kompleks [2]. Hal ini memungkinkan pembuatan tombol kontrol kustom (seperti tombol "Start Eksperimen") yang dapat memanggil *script* eksekusi di *backend* secara langsung.
2. **Ekosistem Luas:** Memiliki dukungan *library* pihak ketiga yang masif, mulai dari komponen UI hingga *library* grafik/plot yang dibutuhkan untuk visualisasi data performa.

Kekurangan framework React.js adalah:

1. ***Learning Curve* Tinggi:** Membutuhkan waktu pembelajaran yang cukup lama untuk menguasai konsep seperti *state management* dan *lifecycle* komponen agar dapat mengembangkan website yang utuh.
2. **Setup Kompleks:** Memerlukan konfigurasi awal yang rumit menggunakan *build tools* (seperti Webpack atau Vite) dan membutuhkan implementasi *backend* terpisah untuk berfungsi penuh.

React.js sangat mendukung pemenuhan Persyaratan Fungsional 4, yaitu menyediakan antarmuka interaktif untuk memulai dan menghentikan eksperimen. Fleksibilitasnya memungkinkan integrasi logika kontrol yang tidak dimiliki oleh *tool* visualisasi murni. Selain itu, kemampuan *rendering* yang efisien mendukung spesifikasi pembaruan data visualisasi ≤ 1 detik, memastikan dashboard menampilkan status *node* secara *real-time*.

**Alternatif framework 2: Grafana**

Grafana adalah platform observabilitas dan visualisasi data *open-source* yang dirancang untuk memonitor metrik infrastruktur dan data *time-series* dari berbagai sumber data [3].

Keunggulan framework Grafana adalah:

1. **Visualisasi *Plug-n-Play*:** Menyediakan fitur visualisasi instan dimana pengguna cukup menghubungkan sumber data (misalnya InfluxDB) tanpa perlu melakukan *coding* tampilan dari nol.
2. **Integrasi Mudah:** Memiliki dukungan *native* untuk berbagai *database* dan *backend* metrik populer yang umum digunakan dalam monitoring jaringan [4].

Kekurangan framework Grafana adalah:


© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.

1. **Kustomisasi Terbatas:** Grafana tidak memberikan kebebasan penuh untuk memodifikasi komponen halaman. Fungsinya terbatas pada *monitoring* (baca data) dan tidak dirancang untuk mengirim perintah kontrol (tulis/eksekusi) ke sistem.

Grafana sangat kuat dalam memenuhi Subobjektif 4 terkait visualisasi metrik performa (throughput, latensi, CPU usage). Namun, Grafana memiliki keterbatasan dalam memenuhi Persyaratan Fungsional 3 (mengatur *deployment* dan menjalankan eksperimen), karena tidak memiliki fitur bawaan untuk eksekusi logika orkestrasi atau intervensi aktif ke *node* uji.

### 2.1.2. Subsistem Eksperimen eBPF/XDP

Subsistem ini adalah inti dari penelitian untuk mencapai **Subobjektif 1** (Implementasi *In-Kernel Fast Path*) dan **Subobjektif 2** (Minimal Overhead). Pemilihan *toolchain* di sini menentukan apakah sistem dapat mencapai target *throughput* ≥ 55% dari DPDK.

**<u>Alternatif Library 1: BCC</u>**

BCC adalah *toolkit* generasi pertama untuk pengembangan eBPF yang menyediakan *wrapper* Python (atau Lua) untuk *user-space*. Kode eBPF (C) ditulis sebagai *string* di dalam kode Python dan dikompilasi saat program dijalankan (*runtime compilation*) [5].

Keunggulan library BCC adalah:

1. **Prototyping Cepat:** Sangat mudah digunakan untuk menulis skrip *tracing* atau *networking* sederhana secara cepat karena memanfaatkan kemudahan bahasa Python.
2. **Ekosistem Tools:** Menyediakan banyak contoh *tools* siap pakai untuk observabilitas sistem.

Kekurangan library BCC adalah:

2. ***Overhead* Dependensi Tinggi:** Mengharuskan setiap *node* di *testbed* memiliki instalasi *compiler* lengkap (LLVM, Clang, Kernel Headers) yang memakan ruang penyimpanan besar [6].
3. ***Startup* Lambat:** Proses kompilasi *on-the-fly* memakan CPU dan memori signifikan saat program dimulai, yang dapat mengganggu presisi pengukuran awal eksperimen

Meskipun BCC memudahkan implementasi fungsi dasar (Subobjektif 1), *overhead* kompilasi saat *runtime* berisiko melanggar Constraint Subobjektif 2 (Minimal Overhead). Konsumsi sumber daya yang tinggi saat inisialisasi dapat mempengaruhi pembacaan metrik CPU *usage* yang seharusnya rendah, sehingga kurang ideal untuk sistem produksi yang membutuhkan efisiensi maksimal.

**<u>Alternatif Library 2: LibBPF (C/Native)</u>**

LibBPF adalah *library* C standar yang dikelola sebagai bagian dari *source tree* kernel Linux. Library ini mendukung teknologi CO-RE (*Compile Once - Run Everywhere*), yang memungkinkan program eBPF dikompilasi sekali menjadi biner dan dijalankan di berbagai versi kernel tanpa kompilasi ulang [6].

Keunggulan library LibBPF adalah:

1. ***High-Performance* & Ringan:** Program dikompilasi menjadi *bytecode* biner kecil yang efisien, meminimalkan *overhead* sistem dan menghilangkan kebutuhan *compiler* di *node* target [7].


© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.

2. **Portabilitas (CO-RE):** Memudahkan pemindahan program antar *node* dengan versi kernel berbeda tanpa perlu kompilasi ulang atau instalasi *header* kernel yang berat

Kekurangan library LibBPF adalah:

1. **Kompleksitas Tinggi:** Menggunakan bahasa C murni yang memerlukan manajemen memori manual dan pemahaman mendalam tentang struktur kernel, sehingga *learning curve*-nya lebih curam dibanding BCC

LibBPF adalah kandidat utama untuk memenuhi Subobjektif 2 (Meminimalkan overhead). Karakteristiknya yang ringan dan efisien mendukung pencapaian syarat pemenuhan CPU usage < 5% saat idle. Selain itu, performa tinggi dari kode *native* C sangat krusial untuk mengejar target throughput ≥ 55% dari baseline DPDK pada skenario beban tinggi.

### 2.1.3. Subsistem Orkestrasi dan Otomasi Testbed Multi-Node

Subsistem ini bertanggung jawab memenuhi Subobjektif 3, yaitu otomatisasi penuh tanpa intervensi manual. Metode yang dipilih harus mampu mencapai target DTRR ≥ 78%.

**Alternatif Tools 1: Ansible**

Ansible adalah *framework* otomasi *open-source* yang menggunakan *playbook* berbasis YAML untuk mendeskripsikan konfigurasi sistem. Ansible bekerja dengan model *agentless*, berinteraksi dengan *node* target melalui protokol SSH standar [8].

Keunggulan tools Ansible adalah:

1. ***Idempotency*:** Menjamin konsistensi; menjalankan *playbook* berulang kali akan selalu menghasilkan status sistem akhir yang sama. Ini sangat penting untuk *error recovery* otomatis dalam eksperimen.
2. ***Agentless*:** Tidak membebani *node* eksperimen dengan *software agen* tambahan, menjaga validitas pengukuran performa eBPF.
3. ***Readability*:** Format YAML yang deklaratif mudah dibaca dan dipelihara.

Kekurangan library Ansible adalah:

1. ***Overhead* Eksekusi:** Relatif lebih lambat dibandingkan eksekusi skrip langsung karena adanya proses pengumpulan fakta sistem (*gathering facts*) dan transfer modul Python di setiap tugas [9].

Ansible paling kuat dalam memenuhi Constraint Subobjektif 3 (Otomatisasi Penuh). Sifat *idempotency*-nya menjamin *repeatability* eksperimen yang tinggi. Meskipun memiliki sedikit *overhead* waktu eksekusi, Ansible tetap mampu mencapai target efisiensi waktu yang signifikan dibandingkan proses manual, serta menjamin stabilitas konfigurasi di lingkungan multi-node.

**Alternatif Tools 2: Shell Scripting (Bash via SSH)**

Metode ini menggunakan skrip Bash *native* yang berisi urutan perintah *shell* imperatif. Skrip dieksekusi dari satu node pengendali ke node target secara sekuensial atau paralel melalui koneksi SSH.

Keunggulan tools Shell Scripting adalah:


© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.

4. **Performa Eksekusi Tinggi:** Memiliki *overhead* paling rendah dan waktu eksekusi tercepat karena berjalan langsung di level OS [10].
5. **Dependensi Minimal:** Hanya membutuhkan terminal Bash dan SSH yang sudah tersedia di semua distribusi Linux.

Kekurangan tools Shell Scripting adalah:

1. **Tidak Idempoten:** Bersifat imperatif; jika skrip gagal di tengah jalan, sistem bisa tertinggal dalam kondisi rusak (*broken state*). Menjalankan ulang skrip tanpa penanganan *error* yang cermat dapat menyebabkan duplikasi atau kegagalan fatal.
2. **Sulit Dipelihara:** Skrip cenderung menjadi panjang dan sulit di-*debug* untuk logika orkestrasi yang kompleks

Metode ini unggul dalam hal kecepatan eksekusi (mendukung metrik DTRR), namun memiliki risiko tinggi terhadap Constraint Subobjektif 3 ("tanpa intervensi manual"). Jika terjadi kegagalan parsial, intervensi manual seringkali diperlukan untuk memperbaiki status *node*, yang bertentangan dengan tujuan otomatisasi penuh.

**Alternatif Tools 3: Python (Library Fabric)**

Fabric adalah *library* Python tingkat tinggi (*high-level*) yang dirancang untuk menyederhanakan eksekusi perintah *shell* dari jarak jauh melalui protokol SSH [11]. Berbeda dengan sekadar menggunakan terminal SSH manual, Fabric membungkus protokol komunikasi tersebut dengan kekuatan bahasa pemrograman Python, memungkinkan pengembang untuk menulis skrip orkestrasi yang cerdas, terstruktur, dan dapat digunakan kembali (*reusable*) untuk mengelola *testbed* jaringan yang kompleks.

Keunggulan tools Python (Library Fabric) adalah:

1. **Orkestrasi Logika Cerdas:** Memungkinkan implementasi alur eksperimen yang dinamis menggunakan fitur bahasa Python (seperti *conditional* `if/else`, *looping*, dan penanganan *exception*). Contohnya, sistem dapat secara otomatis memutuskan untuk mengulang eksperimen jika *throughput* yang terukur di bawah ambang batas, sebuah logika yang sulit dilakukan dengan SSH manual atau skrip Bash sederhana [11].
2. **Pemrosesan Data Langsung (*Data Extraction*):** Output mentah dari perintah terminal (seperti log `bpftool` atau `iperf`) dapat langsung ditangkap, di-*parsing*, dan diubah menjadi format data terstruktur (JSON/CSV) dalam satu skrip yang sama sebelum dikirim ke *dashboard*, menghilangkan kebutuhan akan pemrosesan manual.
3. **Manajemen Multi-Node Sinkron:** Memudahkan sinkronisasi perintah antar *node* yang berbeda (misalnya: menyalakan *server* di Node A terlebih dahulu, menunggu hingga siap, baru kemudian memerintahkan Node B untuk mengirim trafik), memastikan urutan eksekusi yang presisi.

Kekurangan tools Python (Library Fabric) adalah:

1. **Bersifat Imperatif (Non-Idempoten):** Tidak seperti Ansible yang berbasis status (*state-based*), Fabric mengeksekusi perintah secara prosedural. Pengembang harus menulis logika tambahan secara manual untuk menangani pengecekan status (misalnya: "cek apakah *file* sudah ada sebelum membuatnya") guna mencegah *error* saat skrip dijalankan ulang.
2. ***Overhead* Pengembangan Awal:** Membutuhkan waktu penulisan kode (*coding*) yang lebih banyak di awal dibandingkan sekadar menjalankan perintah SSH satu per satu,


© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.

karena pengembang harus membangun struktur skrip dan fungsi penanganan *error* sendiri.

Metode ini sangat krusial untuk memenuhi Subobjektif 3 (Otomatisasi Penuh) dan Subobjektif 4 (Visualisasi). Fabric berperan sebagai "orkestrator" yang menjembatani komunikasi antar *node* dengan logika yang kuat, memastikan *pipeline* eksperimen berjalan otomatis tanpa intervensi manual (memenuhi target DTRR ≥ 78%.). Selain itu, kemampuan *parsing* datanya memastikan metrik performa dapat dikumpulkan dan disajikan ke dashboard secara akurat dan *real-time*.

### 2.1.4. Sistem Terintegrasi

Berdasarkan analisis alternatif metode pada ketiga subsistem utama yang telah diuraikan sebelumnya, langkah selanjutnya adalah merumuskan kandidat desain sistem terintegrasi. Kandidat ini dibentuk dengan mengombinasikan satu alternatif metode dari setiap subsistem, sehingga menghasilkan berbagai variasi arsitektur sistem yang utuh.

Tabel 2. Kombinasi alternatif desain sistem terintegrasi

| No. | Alternatif Desain Sistem Terintegrasi |
|---|---|
| 1. | React.js – BCC – Ansible |
| 2. | React.js – BCC – Shell Scripting |
| 3. | React.js – BCC – Python (Fabric) |
| 4. | React.js – LibBPF – Ansible |
| 5. | React.js – LibBPF – Shell Scripting |
| 6. | React.js – LibBPF – Python (Fabric) |
| 7. | Grafana – BCC – Ansible |
| 8. | Grafana – BCC – Shell Scripting |
| 9. | Grafana – BCC – Python (Fabric) |
| 10. | Grafana – LibBPF – Ansible |
| 11. | Grafana – LibBPF – Shell Scripting |
| 12. | Grafana – LibBPF – Python (Fabric) |
| 13. | React.js + Grafana – BCC – Ansible |
| 14. | React.js + Grafana – Shell Scripting |
| 15. | React.js + Grafana – Python (Fabric) |
| 16. | React.js + Grafana – LibBPF – Ansible |
| 17. | React.js + Grafana – LibBPF – Shell Scripting |
| 18. | React.js + Grafana – LibBPF – Python (Fabric) |

## 2.2. Estimasi Biaya Produk

Bagian ini menguraikan perkiraan biaya riil yang dibutuhkan untuk merealisasikan desain sistem. Pengerjaan dilakukan sepenuhnya oleh tim mahasiswa sebagai bagian dari tugas akhir


© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.

(tanpa biaya jasa pengembangan) dan infrastruktur perangkat keras menggunakan fasilitas laboratorium. Estimasi biaya dibagi menjadi biaya material dan hardware serta biaya operasional listrik, cloud, dan hosting.

### 2.2.1. Komponen Biaya

Rincian komponen biaya adalah sebagai berikut:

1. **Biaya Material & Hardware (Rp 61.853.000):**

   Diasumsikan pembangunan *testbed* membutuhkan 3 unit PC/Server (DUT, Tester, Controller) dan perangkat jaringan pendukung.

   - NIC 5G Ethernet Network Card (Intel E810 Controller, Dual Port SFP28). Estimasi Rp 3.171.000 × 3 unit = Rp 9.513.000.
   - Server/Mini PC (High Spec): ASRock DeskMeet B660 (Intel Core i7-13700K, RAM 32GB, GTX 1650). Estimasi harga Rp 16.780.000/unit × 3 unit = Rp 50.340.000.
   - Switch & Kabel: Kabel DAC/SFP+ dan Switch Pendukung. Estimasi Rp 2.000.000.
   - **Total Biaya Material & Hardware: Rp 61.853.000.**

2. **Biaya Operasional Listrik, Cloud, & Hosting (Rp 3.750.000):**

   - Mengingat spesifikasi server i7-13700K yang berkinerja tinggi, diasumsikan daya rata-rata total sistem (3 Node) adalah 400 Watt, menyala 24 jam.
     - Konsumsi: 0.4 kW × 24 jam × 30 hari = 288 kWh/bulan.
     - Tarif Listrik (Batas Daya R2/R3): ~Rp 1.700/kWh.
     - Biaya: 288 kWh × Rp 1.700 ≈ Rp 500.000/bulan.
     - Total 6 Bulan: Rp 3.000.000.
   - Biaya ini diperlukan untuk men-*deploy* subsistem dashboard ke internet (VPS) agar dapat diakses dan memonitor eksperimen secara *remote*, serta pembelian nama domain.
     - Sewa VPS: Spesifikasi 2 vCPU/4GB RAM. Estimasi Rp 100.000/bulan × 6 bulan (durasi pengerjaan) = Rp 600.000.
     - Registrasi Domain (.id/.com): Biaya langganan 1 tahun = Rp 150.000.
   - **Total Biaya Operasional Listrik, Cloud, & Hosting: Rp 3.750.000**

### 2.2.2. Estimasi Biaya per Alternatif Sistem Terintegrasi

Tabel 3 memperlihatkan perhitungan estimasi biaya total. Karena seluruh alternatif menggunakan infrastruktur *hosting* yang sama, biaya yang dibutuhkan seragam untuk semua kombinasi.

Tabel 3. Estimasi Biaya Produk untuk Setiap Alternatif

| No. | Alternatif Sistem Terintegrasi | Biaya Hardware | Biaya Operasional | Total Estimasi Biaya |
|---|---|---|---|---|
| 1. | React.js – BCC – Ansible | Rp 61.853.000 | Rp 3.750.000 | **Rp 65.603.000** |


© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.

| | | | | |
|---|---|---|---|---|
| 2. | React.js – BCC – Shell Scripting | Rp 61.853.000 | Rp 3.750.000 | **Rp 65.603.000** |
| 3. | React.js – BCC – Python (Fabric) | Rp 61.853.000 | Rp 3.750.000 | **Rp 65.603.000** |
| 4. | React.js – LibBPF – Ansible | Rp 61.853.000 | Rp 3.750.000 | **Rp 65.603.000** |
| 5. | React.js – LibBPF – Python (Fabric) | Rp 61.853.000 | Rp 3.750.000 | **Rp 65.603.000** |
| 6. | Grafana – BCC – Ansible | Rp 61.853.000 | Rp 3.750.000 | **Rp 65.603.000** |
| 7. | Grafana – LibBPF – Shell Scripting | Rp 61.853.000 | Rp 3.750.000 | **Rp 65.603.000** |
| 8. | Grafana – LibBPF – Python (Fabric) | Rp 61.853.000 | Rp 3.750.000 | **Rp 65.603.000** |
| 9. | React.js + Grafana – BCC – Python | Rp 61.853.000 | Rp 3.750.000 | **Rp 65.603.000** |
| 10. | React.js + Grafana – LibBPF – Ansible | Rp 61.853.000 | Rp 3.750.000 | **Rp 65.603.000** |
| 11. | React.js + Grafana – LibBPF – Shell | Rp 61.853.000 | Rp 3.750.000 | **Rp 65.603.000** |
| 12. | Grafana – LibBPF – Python (Fabric) | Rp 61.853.000 | Rp 3.750.000 | **Rp 65.603.000** |
| 13. | React.js + Grafana – BCC – Ansible | Rp 61.853.000 | Rp 3.750.000 | **Rp 65.603.000** |
| 14. | React.js + Grafana – Shell Scripting | Rp 61.853.000 | Rp 3.750.000 | **Rp 65.603.000** |
| 15. | React.js + Grafana – Python (Fabric) | Rp 61.853.000 | Rp 3.750.000 | **Rp 65.603.000** |
| 16. | React.js + Grafana – LibBPF – Ansible | Rp 61.853.000 | Rp 3.750.000 | **Rp 65.603.000** |
| 17. | React.js + Grafana – LibBPF – Shell Scripting | Rp 61.853.000 | Rp 3.750.000 | **Rp 65.603.000** |
| 18. | React.js + Grafana – LibBPF – Python (Fabric) | Rp 61.853.000 | Rp 3.750.000 | **Rp 65.603.000** |

### 2.2.3. Kesimpulan Biaya

Berdasarkan estimasi di atas, total biaya riil yang dibutuhkan untuk merealisasikan desain sistem ini adalah **Rp 65.603.000**. Komponen terbesar dari biaya ini dialokasikan untuk pengadaan infrastruktur perangkat keras (*hardware*) berkinerja tinggi dan biaya operasional energi listrik yang krusial untuk menjaga reliabilitas *testbed* 24 jam.

## 2.3. Analisis Finansial


© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.

Analisis finansial dilakukan untuk mengetahui apakah subsistem dashboard yang dikembangkan layak diimplementasikan secara ekonomis. Analisis ini menggunakan metode BEP dalam satuan waktu untuk menghitung berapa lama waktu yang dibutuhkan agar total biaya investasi infrastruktur *high-end* tersebut dapat tertutupi oleh pendapatan layanan.

Berdasarkan estimasi rincian pada Subbab 2.2, total kebutuhan dana awal (*Initial Investment*) mencakup pengadaan infrastruktur perangkat keras (*hardware*) berkinerja tinggi serta biaya operasional selama masa pengembangan (6 bulan):

Rincian total biaya awal adalah sebagai berikut:

1. Biaya Material & Hardware: Rp 61.853.000
2. Biaya Operasional Pengembangan (6 Bulan): Rp 3.750.000

Total Biaya Awal (Total Investment):

$$Rp\ 61.853.000 + \text{Rp}\ 3.750.000 = Rp\ 65.603.000$$

Layanan ini kemudian ditawarkan kepada pengguna dengan biaya berlangganan sebesar Rp 4.400.000,00 per bulan. Nilai tersebut menjadi *cash inflow* utama dalam analisis finansial.

Perhitungan BEP dilakukan berdasarkan formula dasar berikut:

$$BEP\ (bulan) = \frac{Biaya\ Awal}{Pendapatan\ Bersih\ Bulanan}$$

Dengan memasukkan nilai-nilai yang digunakan pada proyek ini:

$$BEP\ (bulan) = \frac{65.603.000}{4.400.000}$$

$$= 14{,}91\ Bulan\ \approx 1\ Tahun\ 3\ Bulan$$

Berdasarkan hasil perhitungan di atas, titik impas (*Break-Even Point*) diproyeksikan tercapai dalam waktu **14,91 bulan** (sekitar 1 tahun 3 bulan) setelah layanan beroperasi secara penuh.

Waktu pengembalian modal ini masih tergolong **sangat layak (*feasible*)**, terutama mengingat spesifikasi perangkat keras yang digunakan (Intel Core i7 Gen-13 dan 3 unit NIC 25G) memiliki masa pakai ekonomis 3-5 tahun. Hal ini menunjukkan bahwa sistem memiliki profitabilitas yang tinggi, dimana setelah bulan ke-15, pendapatan layanan akan menjadi keuntungan murni bagi penyedia layanan/laboratorium.

## 2.4. Penentuan Solusi Desain Terbaik

### 2.4.1. Pembobotan Kategori dari Alternatif Metode Desain

Untuk menentukan solusi desain terbaik, digunakan kategori dari alternatif metode desain yang telah disusun pada Tabel 2. Berikut ini adalah pembobotan yang ditentukan.

Tabel 3. Pembobotan pada Kategori dari Alternatif Metode Desain.

| Kategori | Bobot |
|---|---|
| Framework Eksperimen eBPF | 5 |
| Program Automasi Eksperimen | 4 |
| Framework Pengembangan Dashboard | 3 |


© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.

Sebelum dilakukan perhitungan pada matriks keputusan, setiap metode dari masing-masing kategori diberikan nilai bobot (skor) berdasarkan performa, fleksibilitas, dan kesesuaiannya terhadap persyaratan desain. Skala penilaian yang digunakan adalah sebagai berikut.

Tabel 4. Kriteria Penilaian Alternatif Metode Desain.

| Kategori | Metode Alternatif | Nilai | Alasan Penilaian |
|---|---|---|---|
| Framework Eksperimen eBPF | BCC | 1 | Mudah dipelajari (*prototyping* cepat), namun memiliki *overhead* dependensi tinggi dan waktu *startup* lambat. |
| | LibBPF | 2 | Mendukung konsep CO-RE (*Compile Once - Run Everywhere*), ringan, dan memiliki performa *native* yang tinggi sesuai dengan *constraint* efisiensi. |
| Program Automasi | Ansible | 1 | Stabil dan idempoten, namun memiliki *overhead* eksekusi yang lambat dan kurang fleksibel untuk logika eksperimen dinamis. |
| | Bash Script | 2 | Eksekusi sangat cepat (*native*), namun sulit dipelihara (*maintainability* rendah) dan tidak memiliki *error handling* canggih. |
| | Python (Fabric) | 3 | Menawarkan keseimbangan terbaik antara kontrol logika yang kompleks, kemudahan pengolahan data (*parsing*), dan kecepatan eksekusi. |
| Framework Dashboard | React.js | 1 | Fleksibilitas tinggi untuk kontrol, namun membutuhkan waktu pengembangan lama (*setup* kompleks) untuk visualisasi data. |
| | Grafana | 2 | Sangat kuat dalam visualisasi data *time-series*, namun terbatas dalam fungsi kontrol/interaksi aktif ke sistem. |
| | React.js + Grafana | 3 | Solusi hibrida yang menggabungkan keunggulan kontrol interaktif (React) dan kemudahan visualisasi data (Grafana), memenuhi seluruh spesifikasi fungsional. |

### 2.4.2. Matriks Keputusan

Dari berbagai alternatif desain sistem terintegrasi, diputuskan metode desain yang terbaik melalui matriks keputusan, dimana setiap alternatif desain diberikan bobot nilai antara 1 – 5 seperti yang tertera pada Tabel 4. Berikut adalah matriks keputusan dari alternatif desain.

Tabel 5. Matriks Keputusan Alternatif Desain Sistem Terintegrasi.

| **Kategori** | **Nama** | **Framework Eksperimen eBPF** | **Program Automasi Eksperimen** | **Framework Pengembangan Dashboard** | **Total Bobot** |
|---|---|---|---|---|---|
| | **Bobot** | **10** | **8** | **6** | |
| **Alternatif 1** | | 10 | 8 | 6 | **24** |
| **Alternatif 2** | | 10 | 8 | 12 | **30** |
| **Alternatif 3** | | 10 | 8 | 18 | **36** |
| **Alternatif 4** | | 10 | 16 | 6 | **32** |
| **Alternatif 5** | | 10 | 16 | 12 | **38** |
| **Alternatif 6** | | 10 | 16 | 18 | **44** |
| **Alternatif 7** | | 20 | 8 | 6 | **34** |


© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.

| | | | | |
|---|---|---|---|---|
| **Alternatif 8** | 20 | 8 | 12 | **40** |
| **Alternatif 9** | 20 | 8 | 18 | **46** |
| **Alternatif 10** | 20 | 16 | 6 | **42** |
| **Alternatif 11** | 20 | 16 | 12 | **48** |
| **Alternatif 12** | 20 | 16 | 18 | **54** |
| **Alternatif 13** | 30 | 8 | 6 | **44** |
| **Alternatif 14** | 30 | 8 | 12 | **50** |
| **Alternatif 15** | 30 | 8 | 18 | **56** |
| **Alternatif 16** | 30 | 16 | 6 | **52** |
| **Alternatif 17** | 30 | 16 | 12 | **58** |
| **Alternatif 18** | 30 | 16 | 18 | **64** |

Dari hasil penilaian setiap alternatif sistem terintegrasi, alternatif desain yang dipilih adalah alternatif 18. Metode 18 merupakan kombinasi antara framework dashboard React.Js + Grafana, framework eksperimen LibBPF, dan framework automasi sistem Python.

## 3. RENCANA PELAKSANAAN PROYEK DESAIN

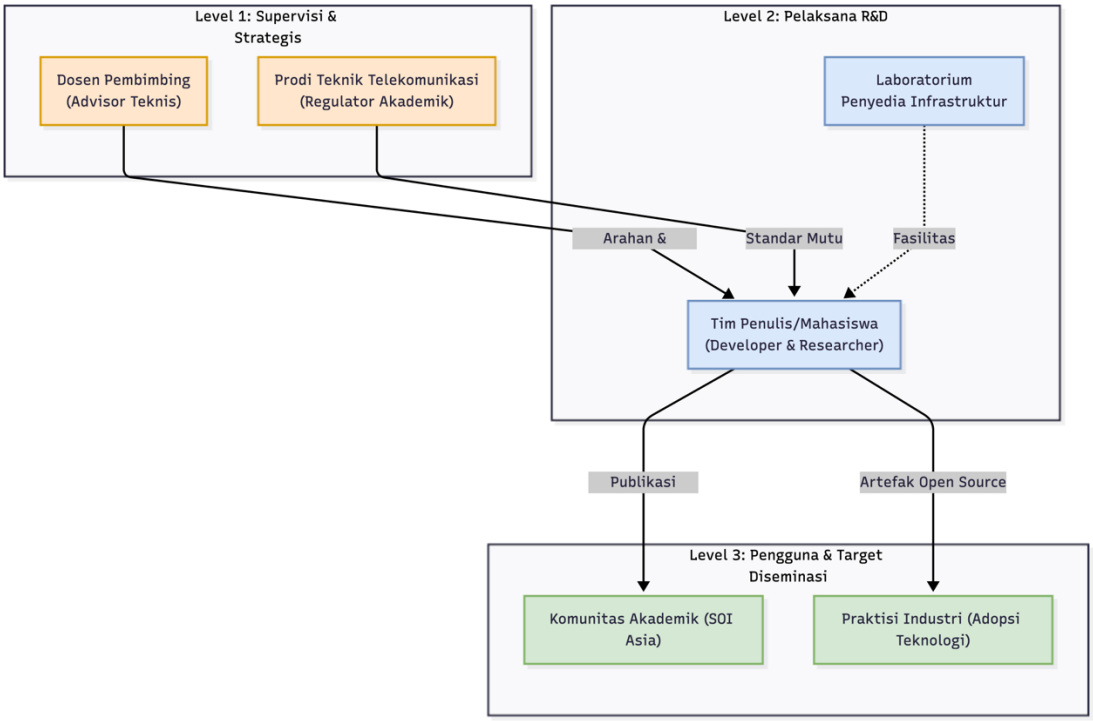
### 3.1. Sumber dan Cara Mendapatkan Dana Pembuatan Desain

Berdasarkan alternatif desain terpilih (Alternatif 18: React.js + Grafana, LibBPF, dan Python), strategi pendanaan proyek ini dibagi menjadi tiga tahapan sebagai berikut:

1. **Tahap Inisiasi dan Pengembangan Awal (Dana Awal)**
   Dana awal untuk realisasi purwarupa sistem ini diperoleh melalui kombinasi dana swadaya tim pengembang dan dukungan fasilitas institusi. Dana swadaya digunakan untuk membiayai kebutuhan operasional *hosting* dashboard (Cloud VPS dan Domain). Sementara itu, dukungan fasilitas utama berupa infrastruktur perangkat keras (*testbed server*, *switch*, dan jaringan) disediakan oleh Laboratorium Teknik Telekomunikasi ITB, sehingga mengeliminasi kebutuhan modal awal yang besar untuk pengadaan aset fisik.
2. **Tahap Implementasi Produk Komersial**
   Untuk memasuki fase hilirisasi menuju produk yang siap dipakai di lingkungan industri, seluruh kebutuhan pendanaan akan dipenuhi melalui dukungan internal dari STEI ITB. Dukungan ini mencakup pembiayaan pengembangan lanjutan, pengujian skala terbatas, penyediaan infrastruktur laboratorium, serta fasilitas yang diperlukan untuk memastikan bahwa sistem dapat mencapai tingkat kesiapan teknologi yang layak diadopsi oleh pengguna eksternal.
3. **Tahap Keberlanjutan (Sustainability)**
   Akhirnya, untuk memastikan keberlanjutan pengembangan produk jangka panjang, sumber dana utama akan mengandalkan margin keuntungan dari model bisnis layanan. Mengingat perangkat lunak inti berbasis *Open Source*, pendapatan diperoleh melalui model SaaS untuk dashboard monitoring atau jasa konsultasi profesional (*professional services*) untuk implementasi sistem di lingkungan industri/kampus mitra. Dengan demikian, arus kas mandiri ini dapat menjamin biaya pemeliharaan *server* dan pengembangan fitur lanjutan secara berkelanjutan.

### 3.2. Struktur Organisasi Pengembangan dan Pemasaran Desain


© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.




Gambar 1. Struktur Organisasi.

Dalam konteks penelitian R&D (*Research and Development*) untuk implementasi teknologi *in-kernel fast path* ini, struktur organisasi disusun berdasarkan fungsi akademis dan teknis. Struktur ini bertujuan memastikan validitas metodologi, keberhasilan eksekusi teknis, dan efektivitas penyebaran (diseminasi) hasil penelitian. Organisasi dibagi menjadi tiga level sebagai berikut:

- **Level 1: Kewenangan Strategis & Supervisi (Strategic Level)**
  Level ini memiliki kewenangan tertinggi dalam memberikan arahan, menyetujui metodologi, dan memvalidasi kelayakan hasil akhir.
  - **Institusi & Regulator:** Program Studi Teknik Telekomunikasi ITB (Menetapkan standar kelayakan akademik).
  - **Advisor (Pembimbing Utama):** Dosen Pembimbing (Memberikan arahan strategis terkait arah riset, memvalidasi metode pengujian, dan menyetujui target performa sistem).
- **Level 2: Pelaksana Riset & Pengembangan (Operational Level)**
  Level ini adalah inti dari kegiatan proyek yang bertanggung jawab atas seluruh aspek teknis dan operasional.
  - **Lead Researcher & Developer:** Tim Penulis (Mahasiswa). Bertanggung jawab penuh atas:
    - **Engineering:** Implementasi kode eBPF/XDP dan skrip otomasi Python.
    - **Testing:** Pembangunan *testbed* 25Gbps dan pelaksanaan pengujian performa.
    - **Reporting:** Analisis data dan penyusunan dokumen teknis.
  - **Fasilitator Teknis:** Laboratorium Teknik Telekomunikasi (Menyediakan akses ke infrastruktur testbed dan perangkat jaringan).
- **Level 3: Penerima Manfaat & Target Diseminasi (User Level)**
  Level ini adalah pihak yang akan memanfaatkan hasil studi dan artefak teknologi yang dihasilkan. "Pemasaran" pada level ini dilakukan dalam bentuk publikasi ilmiah dan kontribusi *open source*.


© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.

- **Komunitas Akademik:** Platform testbed ini dirancang terutama untuk meningkatkan kemampuan penelitian dan pembelajaran bagi komunitas SOI Asia. Melalui infrastruktur eksperimen yang terstandarisasi dan mudah diakses, para peneliti dan mahasiswa dapat melakukan pengujian, validasi, serta pengembangan skenario jaringan secara lebih terukur dan efisien..
- **Praktisi Industri/Network Engineers:** Industri telekomunikasi dan pusat data merupakan pihak yang berpotensi memperoleh manfaat paling besar dari penelitian ini. Melalui hasil pengujian yang terdokumentasi dalam publikasi teknis dan ilmiah, kedua industri tersebut dapat menilai secara lebih akurat kelayakan penerapan eBPF/XDP sebagai alternatif terbuka terhadap solusi proprietary yang selama ini mendominasi.

# 4. KESIMPULAN DAN RENCANA SELANJUTNYA

## 4.1 Kesimpulan

*Design and Implementation of In-Kernel Fast Path Data Plane Using eBPF XDP for High-Performance Networking* dirancang untuk mengatasi permasalahan fundamental pada infrastruktur jaringan berkecepatan tinggi yang mendukung aplikasi berbasis kecerdasan buatan dan pemrosesan data real-time. Identifikasi masalah menunjukkan bahwa *stack* jaringan Linux konvensional memiliki keterbatasan *throughput* dan latensi, sementara solusi alternatif *kernel bypass* berbasis *user-space* seperti DPDK dan VPP, meskipun cepat, menimbulkan inefisiensi sumber daya CPU akibat mekanisme *busy-polling* serta hilangnya fungsionalitas fitur bawaan kernel. Sebagai solusi, penelitian ini mengajukan desain implementasi *in-kernel fast path data plane* menggunakan teknologi eBPF/XDP yang bertujuan menyeimbangkan performa tinggi dengan efisiensi sumber daya dan integritas fitur keamanan kernel.

Perancangan sistem diformulasikan dengan objektif teknis yang terukur, yaitu pencapaian *throughput* minimal 55% dari performa DPDK dan penggunaan CPU yang proporsional terhadap beban trafik. Selain aspek performa jaringan, desain ini juga mencakup pengembangan sistem orkestrasi dan visualisasi terpadu guna mengatasi kompleksitas manajemen eksperimen pada *testbed multi-node*. Target fungsional sistem ditetapkan untuk mampu mengotomatisasi seluruh tahapan pengujian dengan tingkat efisiensi waktu konfigurasi lebih cepat 78% dibandingkan metode manual, serta menyajikan metrik performa secara *real-time* melalui *dashboard* terpusat.

Evaluasi komprehensif terhadap berbagai alternatif metode teknis telah dilakukan melalui matriks keputusan yang mempertimbangkan aspek performa, fleksibilitas, dan kemudahan integrasi. Berdasarkan pembobotan tersebut, ditetapkan solusi desain terbaik berupa integrasi antara framework React.js dan Grafana untuk visualisasi data, penggunaan library LibBPF berbasis C/Native untuk pengembangan program eBPF yang ringan dan portabel, serta pemanfaatan Python Fabric untuk orkestrasi otomatisasi *testbed*. Analisis kelayakan ekonomi dan finansial juga mengonfirmasi bahwa rancangan ini bersifat *feasible* untuk direalisasikan dengan biaya investasi operasional yang rendah dan estimasi waktu pengembalian modal yang singkat, sehingga sistem ini dinyatakan siap untuk memasuki tahap implementasi penuh dan pengujian kinerja pada tahap selanjutnya.


© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.

## 4.2 Rencana Desain Selanjutnya (Tugas Akhir 2)

Pada pelaksanaan Tugas Akhir 2, fokus utama kegiatan akan diarahkan pada realisasi implementasi teknis dari rancangan sistem terintegrasi yang telah disepakati, yaitu kombinasi antara LibBPF, Python, serta React.js dan Grafana. Pengembangan subsistem *in-kernel fast path* akan dilakukan secara mendalam menggunakan *library* LibBPF dengan bahasa C, di mana pemrograman akan difokuskan pada penyusunan logika *packet filtering* dan *forwarding* yang efisien. Implementasi ini juga mencakup verifikasi kompatibilitas program XDP dengan fitur *native firewall* Linux untuk memastikan bahwa fungsi keamanan kernel tetap berjalan beriringan dengan akselerasi paket, sesuai dengan *constraint* desain yang menuntut stabilitas dan keamanan sistem.

Selanjutnya, pengembangan akan dilanjutkan pada sisi otomatisasi sistem dan antarmuka pengguna. Skrip otomatisasi berbasis Python akan dikembangkan untuk menangani manajemen *life-cycle* eksperimen pada *testbed multi-node*, mulai dari penyebaran konfigurasi (*deployment*), eksekusi program eBPF, hingga pengumpulan data log secara otomatis. Bersamaan dengan itu, pembangunan *dashboard* akan dilakukan dengan mengintegrasikan *framework* React.js sebagai pusat kontrol eksperimen dan Grafana sebagai media visualisasi data *time-series*. Integrasi ini bertujuan untuk menyediakan pengalaman pemantauan yang *real-time* dan meminimalisir intervensi manual dalam pengelolaan *testbed*.

Tahap akhir dari Tugas Akhir 2 akan mencakup pengujian performa secara komprehensif dan analisis perbandingan (benchmarking). Serangkaian uji beban trafik tinggi akan dijalankan untuk memvalidasi apakah implementasi eBPF/XDP mampu mencapai target *throughput* minimal 55% dari performa DPDK serta menjaga penggunaan CPU tetap rendah dan proporsional. Selain metrik jaringan, evaluasi terhadap efisiensi operasional juga akan dilakukan untuk membuktikan bahwa pipeline otomatisasi yang dibangun mampu memangkas waktu konfigurasi hingga 78% dibandingkan metode manual. Seluruh hasil pengukuran kemudian akan dianalisis untuk menyimpulkan kelayakan teknis solusi yang diajukan.


© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.

**Referensi**

[1] Meta Open Source, "React – A JavaScript library for building user interfaces," react.dev, 2024.

[2] R. Wieruch, The Road to React: Your journey to master React.js in JavaScript. Independently published, 2023.

[3] Grafana Labs, "Grafana Documentation," *grafana.com*, 2024. [Online].

[4] R. V. B. P. S., & P. J. K., "Real-Time Network Monitoring and Analysis using Grafana and Prometheus," *International Journal of Engineering Research & Technology (IJERT)*, vol. 9, no. 06, 2020.

[5] IO Visor Project, "BCC - Tools for BPF-based Linux IO analysis, networking, monitoring, and more," *GitHub*, 2023. [Online].

[6] A. Nakryiko, "BPF Portability and CO-RE," *Facebook Engineering Blog*, 2020. [Online].

[7] Isovalent, "eBPF - The Future of Networking and Security," *isovalent.com*, 2023.

[8] Red Hat, "Ansible Documentation," *docs.ansible.com*, 2024.

[9] J. Geerling, *Ansible for DevOps: Server and configuration management for humans*. Midwestern Mac, LLC, 2020.

[10] GNU Project, "Bash Reference Manual," *gnu.org*, 2023.

[11] Forcier, "Fabric documentation," *fabfile.org*, 2023.



© 2025 Prodi Teknik Telekomunikasi-ITB. Pengungkapan dan penggunaan seluruh isi dokumen hanya dapat dilakukan atas ijin tertulis Prodi Teknik Telekomunikasi - ITB Jalan Ganesha 10 Bandung, 40132 Indonesia.